

Authentication, Authorization & OAuth

Day 5



Agenda for Day 5

- ASP.NET Core Identity & Authentication Basics
- Cookie-Based Authentication (for MVC)
- JWT Authentication (for Web API)
- Authorization Mechanisms (Role-Based, Policy-Based, User-Specific)
- OAuth 2.0 & External Logins
- Hands-on Demo & Lab Assignment

Authentication vs. Authorization (Review)

- **Authentication:** Verifying who a user is ("Are you who you say you are?").
- Examples: Username/password, biometrics, tokens.
- **Authorization:** Determining what an authenticated user is allowed to do ("What are you allowed to access/do?").
- Examples: Accessing specific resources, performing certain actions (e.g., delete product).

ASP.NET Core Identity

- **Purpose:** A comprehensive membership system for ASP.NET Core applications.
- **Features:** User registration, login, logout, password hashing, two-factor authentication, external logins, role management.
- **Integration:** Built on Entity Framework Core, provides default UI (Razor Pages).

Setup

- 1. Add Identity services to the container (Program.cs)
- AddDefaultIdentity()**
 - Microsoft.AspNetCore.Identity.EntityFrameworkCore
 - Microsoft.AspNetCore.Identity.UI
 - Simple configuration (no roles)
 - Includes UI (MVC only)

```
builder.Services.AddDefaultIdentity<IdentityUser>(options =>
options.SignIn.RequireConfirmedAccount = true)
    .AddRoles<IdentityRole>() // Optional: Add support for roles
    .AddEntityFrameworkStores<ApplicationDbContext>() // Specify your DbContext
    .AddDefaultUI();
```

Setup

- AddIdentity()
 - Microsoft.AspNetCore.Identity.EntityFrameworkCore
 - Microsoft.AspNetCore.Authentication.JwtBearer
 - Full feature
 - MVC or WebAPI

```
builder.Services.AddIdentityCore<IdentityUser>(option=>
{
    option.Password.RequireDigit = false;
    option.User.RequireUniqueEmail = true;
})
    .AddEntityFrameworkStores<CompanyDbContext>()
    .AddApiEndpoints(); // Adds /register, /login endpoints
// Add Authentication (e.g., JWT)
builder.Services.AddAuthentication().AddJwtBearer();
// Add authorization
builder.Services.AddAuthorization();
```

Setup

■ 2. Configure the DbContext for Identity

```
// load connection string from appsettings.json
var connectionString =
builder.Configuration.GetConnectionString("DefaultConnection")

// This assumes ApplicationDbContext inherits from IdentityDbContext<IdentityUser>
builder.Services.AddDbContext<CompanyDbContext>(options =>
    options.UseSqlServer(connectionString));
```

```
public class CompanyDbContext: IdentityDbContext<IdentityUser>
{

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder); // must for IdentityDbContext
    }
}
```

Setup

- 3. Add authentication middleware to the request pipeline

```
// This must be placed before UseAuthorization()  
app.UseAuthentication();  
app.UseAuthorization();
```

- 4. map identity razor pages

```
app.MapRazorPages()  
    .WithStaticAssets();
```


Setup

- 5. add scaffold Identity
- 6. add `<partial name="_LoginPartial" />` in `_Layout`

Setup

- Example of a custom **IdentityUser** (Models folder)

```
public class ApplicationUser:IdentityUser
{
    // Add any custom properties here, e.g.,
    public string FirstName { get; set; }
    public string LastName { get; set; }
}
```

Setup

- Example of a custom `ApplicationDbContext` (Data folder)

```
public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options)
        : base(options)
    {
    }

    // You can add other DbSetes for your application data here
}
```

Cookie-Based Authentication (for MVC)

- **Mechanism:** After successful login, the server issues an encrypted cookie to the client.
- **How it works:**
 - 1. User submits credentials to MVC app.
 - 2. MVC app authenticates user (via **userManager** , **SignInManager**).
 - 3. If successful, MVC app issues an authentication cookie.
 - 4. Browser automatically sends this cookie with subsequent requests.
 - 5. MVC app validates the cookie to identify the user.
 - **[Authorize] attribute:** Applied to MVC controllers/actions to restrict access.

```
[Authorize] // Requires authentication for all actions in this controller
public class HomeController : Controller
{
    public IActionResult Index()
    {
        // This action requires an authenticated user
    }
    [AllowAnonymous] // Allows unauthenticated access to this specific action
    public IActionResult About()
    {
        // This action can be accessed by anyone
    }
    [Authorize(Roles = "Admin")] // Requires the user to be in the 'Admin' role
    public IActionResult AdminDashboard()
    {
        // Only users with the 'Admin' role can access this
    }
    [Authorize(Policy = "RequireManagerRole")] // Requires a specific authorization policy
    public IActionResult ManagerReport()
    {
        // Only users satisfying the 'RequireManagerRole' policy can access this
    }
}
```

JWT Authentication (for Web API)

- **Mechanism:** Uses JSON Web Tokens for stateless authentication.
- **How it works:**
 - 1. Client sends credentials to API login endpoint.
 - 2. API authenticates user, generates a JWT, and returns it.
 - 3. Client stores the JWT (e.g., in local storage).
 - 4. Client sends the JWT in the **Authorization: Bearer <token>** header with each subsequent request.
 - 5. API validates the JWT (signature, expiration, claims) to authenticate the user.
- **Stateless:** No session state on the server, suitable for APIs.

Authorization Mechanisms

- **Role-Based Authorization:**
- Simplest form, based on user's assigned roles (e.g., "Admin", "User", "Manager").
- Usage: `[Authorize(Roles = "Admin")]` or `[Authorize(Roles = "Admin,Manager")]`.

Authorization Mechanisms

- **Policy-Based Authorization:**
- More flexible and powerful.
- Define custom requirements (e.g., "MustBeOver21", "CanEditMovie").
- Implement handlers that check these requirements.
- Usage: `[Authorize(Policy = "MyPolicy")]`.